

K L Deemed to be University

Data Structures and Algorithms - 2

Case Study Question Bank with Solution Key
One Selected Case Study from Each CO | CO1 - CO3

Student Name	G. Akshay
Roll No	2520030363
Subject	DSA-2
Course Code	25SC1305E
Mode	Project-Based Learning
BTL Level	4 - Apply/Analyze
Total Case Studies	3 complete case studies
Marks Pattern	5 + 7 + 3 = 15 Marks each

CO No	Selected Topic	Case Study Theme	BTL
CO1	BST Range Search	Library ISBN shelf lookup	4
CO2	B+ Tree Range Scan	Bank transaction range scan	4
CO3	BFS Shortest Path	Campus shuttle route planning	4

This updated version reduces the original 9-case question bank into a focused 3-case submission: one case study per course outcome. Each case contains a real-world scenario, diagram, three sub-questions, and a detailed solution key with code/pseudocode where required.

CASE STUDY 1 - CO1: Library ISBN Shelf Lookup using BST Range Search

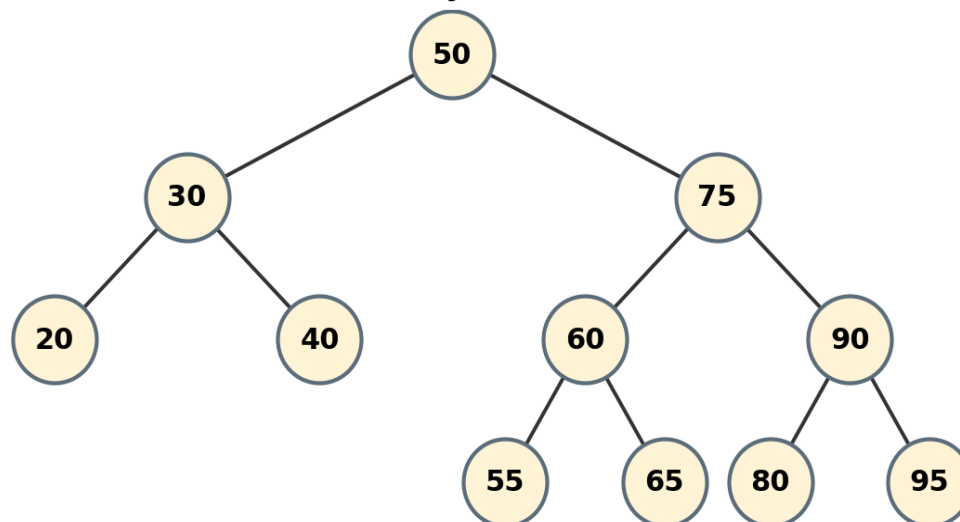
CO	CO1	Topic	BST Range Search
Scenario	Digital library ISBN suffix index	Marks	15
BTL	4	Main Skill	Search, floor/ceil, sorted range output

SCENARIO / CASE STUDY

A digital library stores active book ISBN suffixes in a Binary Search Tree so the librarian can quickly answer shelf queries. The current tree was formed from mixed arrivals, not sorted arrivals. The librarian frequently asks for all ISBN suffixes between two limits and also checks predecessor/successor values when a book is missing.

- Stored keys: 50, 30, 75, 20, 40, 60, 90, 55, 65, 80, 95.
- Range query: [35, 82]. Missing key check: 58.
- The tree must return results in sorted order while avoiding unnecessary subtree traversal.

BST for Library ISBN Suffix Index



Sub	Question	Marks	CO/BTL
(a)	Draw the BST and list the keys visited during search for 58. Find floor(58) and ceil(58).	5	CO1/BTL4
(b)	Write code/pseudocode for rangeSearch(root, L, R) that prints only keys between L and R in sorted order. Apply it for [35, 82].	7	CO1/BTL4
(c)	Analyze the time complexity of search and range query. State whether this tree is suitable for large library indexing.	3	CO1/BTL4

Solution Key - Case Study 1

Answer (a): Search path for 58 is 50 -> 75 -> 60 -> 55 -> null/right. Since 58 is not present, floor(58) = 55 and ceil(58) = 60. The BST property allows every left branch to hold smaller keys and every right branch to hold larger keys.

Answer (b): Range output in sorted order for [35, 82] is: 40, 50, 55, 60, 65, 75, 80. The code prunes left subtrees when root.key <= L and prunes right subtrees when root.key >= R.

```

void rangeSearch(Node root, int L, int R) {
    if (root == null) return;
    if (root.key > L)
        rangeSearch(root.left, L, R);
    if (root.key >= L && root.key <= R)
        print(root.key);
    if (root.key < R)
        rangeSearch(root.right, L, R);
}
  
```

```
if (root.key < R)
    rangeSearch(root.right, L, R);
}
```

Answer (c): Normal search takes $O(h)$, where h is the height of the tree. Range query takes $O(h + k)$, where k is the number of reported keys. For this tree, h is small, so it works well today. For continuously increasing ISBN arrivals, a plain BST can become skewed; AVL or Red-Black Tree indexing would be safer for large library systems.

CASE STUDY 2 - CO2: Bank Transaction Range Scan using B+ Tree

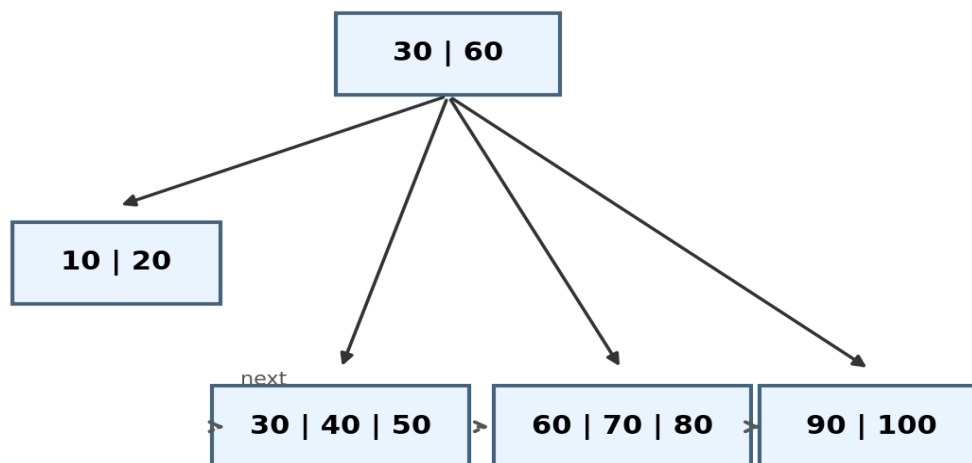
CO	CO2	Topic	B+ Tree Range Scan
Scenario	Bank transaction amount dashboard	Marks	15
BTL	4	Main Skill	Leaf-linked range query and database indexing

SCENARIO / CASE STUDY

A bank dashboard stores transaction amounts in a B+ Tree. Internal nodes guide search, while all actual records are stored in linked leaf nodes. This design is useful for range queries such as listing transactions between two amounts.

- Leaf-level amounts: [10, 20] -> [30, 40, 50] -> [60, 70, 80] -> [90, 100].
- Query range: 35 to 85.
- The solution should use leaf links after locating the first relevant leaf.

B+ Tree Leaf-Level Range Scan Structure



Sub	Question	Marks	CO/BTL
(a)	Explain how search reaches the first relevant leaf for amount 35. List the range output from 35 to 85.	5	CO2/BTL4
(b)	Write code/pseudocode for rangeScan(root, L, R) using the linked leaves of a B+ Tree.	7	CO2/BTL4
(c)	State why B+ Trees are preferred over B-Trees for database range queries.	3	CO2/BTL4

Solution Key - Case Study 2

Answer (a): The search follows internal separators until it reaches the leaf that can contain 35, which is [30, 40, 50]. The output from 35 to 85 is 40, 50, 60, 70, 80.

Answer (b): The algorithm first finds the starting leaf. Then it scans keys in that leaf and follows next pointers until a key greater than R is found.

```

List<Integer> rangeScan(BPlusNode root, int L, int R) {
    Leaf leaf = findLeaf(root, L);
    List<Integer> ans = new ArrayList<>();
    while (leaf != null) {
        for (int key : leaf.keys) {
            if (key > R) return ans;
            if (key >= L) ans.add(key);
        }
        leaf = leaf.next;
    }
    return ans;
}
  
```

```
leaf = leaf.next;  
}  
return ans;  
}
```

Answer (c): B+ Trees keep all records at the leaf level and link leaves sequentially. This makes range scans fast and cache-friendly. B-Trees may store records in internal nodes, so sequential range output is less direct.

CASE STUDY 3 - CO3: Campus Shuttle Route using BFS Shortest Path

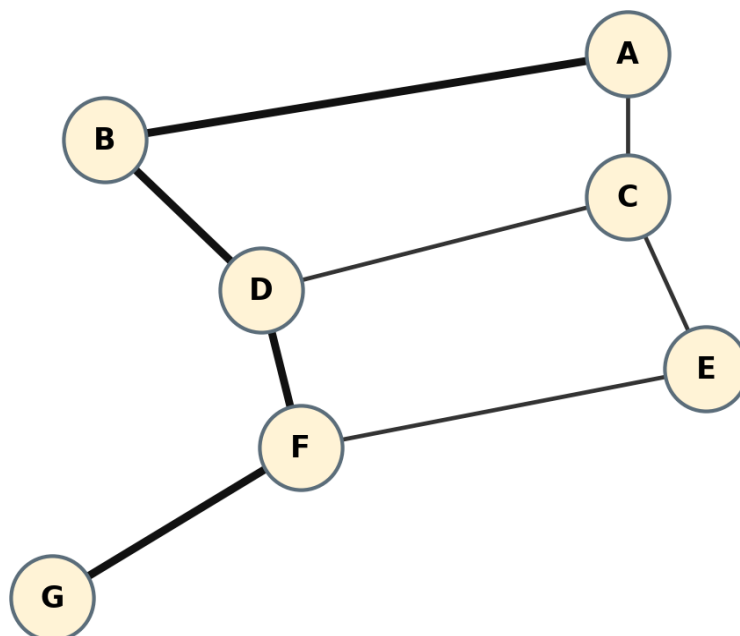
CO	CO3	Topic	BFS Shortest Path
Scenario	Campus shuttle route planning	Marks	15
BTL	4	Main Skill	Minimum-hop path in an unweighted graph

SCENARIO / CASE STUDY

A campus shuttle app models stops as an unweighted graph. Each road segment has equal travel cost for route planning. The app must find the minimum number of stops from the main gate to the hostel area.

- Stops: A, B, C, D, E, F, G.
- Roads: A-B, A-C, B-D, C-D, C-E, D-F, E-F, F-G.
- Start: A. Destination: G.

Campus Shuttle Graph - BFS Shortest Path



Sub	Question	Marks	CO/BTL
(a)	Perform BFS from A. Write the BFS order and shortest path from A to G.	5	CO3/BTL4
(b)	Write BFS code/pseudocode that stores parent[] to reconstruct the shortest path.	7	CO3/BTL4
(c)	State the time complexity and explain why DFS is not ideal for shortest path in an unweighted graph.	3	CO3/BTL4

Solution Key - Case Study 3

Answer (a): One valid BFS order is A, B, C, D, E, F, G. A shortest path can be A -> B -> D -> F -> G or A -> C -> D -> F -> G depending on adjacency order. It uses 4 edges.

Answer (b): BFS uses a queue. When a node is first visited, parent[child] is set to the current node. After reaching the destination, parent links are followed backward to reconstruct the route.

```

List<Node> bfsPath(Graph g, Node src, Node dest) {
    Queue<Node> q = new LinkedList<>();
    Map<Node, Node> parent = new HashMap<>();
    Set<Node> visited = new HashSet<>();
  
```

```
q.add(src);
visited.add(src);
parent.put(src, null);

while (!q.isEmpty()) {
    Node u = q.remove();
    if (u == dest) break;
    for (Node v : g.adj[u]) {
        if (!visited.contains(v)) {
            visited.add(v);
            parent.put(v, u);
            q.add(v);
        }
    }
}
return buildPath(parent, dest);
}
```

Answer (c): BFS runs in $O(V + E)$. It explores the graph layer by layer, so the first time it reaches G is the minimum-edge path. DFS may go deep along a longer route and does not guarantee minimum hops without extra processing.